

Unit 3

SMD Practical + Oral Exam Notes

UNIT 3 — INTERACTION AND BEHAVIOR MODELING

Topic 1: Activity Diagram (CRITICAL FOR PRACTICAL)

1. Simple Exam Definition:

An Activity Diagram is an advanced flowchart that models the **workflow** of a system or process, supporting sequential logic, decisions, and **parallel execution** using Fork and Join.

2. Core Concept Explanation — All Key Elements:

Element	Shape	Description
Initial Node	Solid black circle ●	Every diagram starts here
Activity	Rounded rectangle	A step in the workflow
Decision Node	Diamond ◇	Branches into multiple paths with guard conditions
Merge Node	Diamond ◇	Joins multiple conditional paths back into one
Fork	Thick horizontal/vertical bar (split)	Splits one flow into PARALLEL flows
Join	Thick horizontal/vertical bar (merge)	Waits for ALL parallel flows to complete
Flow Final	Circle with X	Ends one path (not the whole diagram)
Activity Final	Bullseye ⊙	Ends the entire activity
Swimlane	Vertical column (partition)	Shows WHO performs each activity

Guard Expressions:

- Written in square brackets on decision arrows: `[condition = true]`.
- Example: `[seatAvailable = true]` on the "Yes" arrow out of a decision.

Fork vs Join — The Key Difference:

- Fork** = One input, multiple outputs (parallel activities START).
- Join** = Multiple inputs, one output (parallel activities FINISH and synchronize).

3. Real Practical Example (Student Management System — Course Enrollment):

- Student fills form.
- System validates (Decision: `[valid?]`).
- If invalid → show error → back to form.
- If valid → **Fork** → (simultaneously: Deduct Fee **AND** Update Seat Count).
- Join** → Both parallel actions must complete.

6. Send confirmation email → End.

4. Examiner Viva Questions:

- What is the difference between a flowchart and an activity diagram?
- What is a Fork? What is a Join?
- What are Swimlanes?
- What is a Guard Expression?

5. Strong Viva Answers:

- *Answer:* A flowchart only shows sequential and conditional flow. An Activity Diagram additionally supports **parallel execution** using Fork and Join, and can use Swimlanes to show responsibilities.
- *Answer:* A Fork splits one flow into multiple simultaneous (parallel) flows. A Join waits for ALL parallel flows to finish before proceeding.
- *Answer:* Swimlanes divide the diagram into vertical columns showing which actor or component is responsible for each activity.
- *Answer:* A Guard Expression is a boolean condition inside square brackets placed on the arrow leaving a decision node. Example: `[payment successful]`.

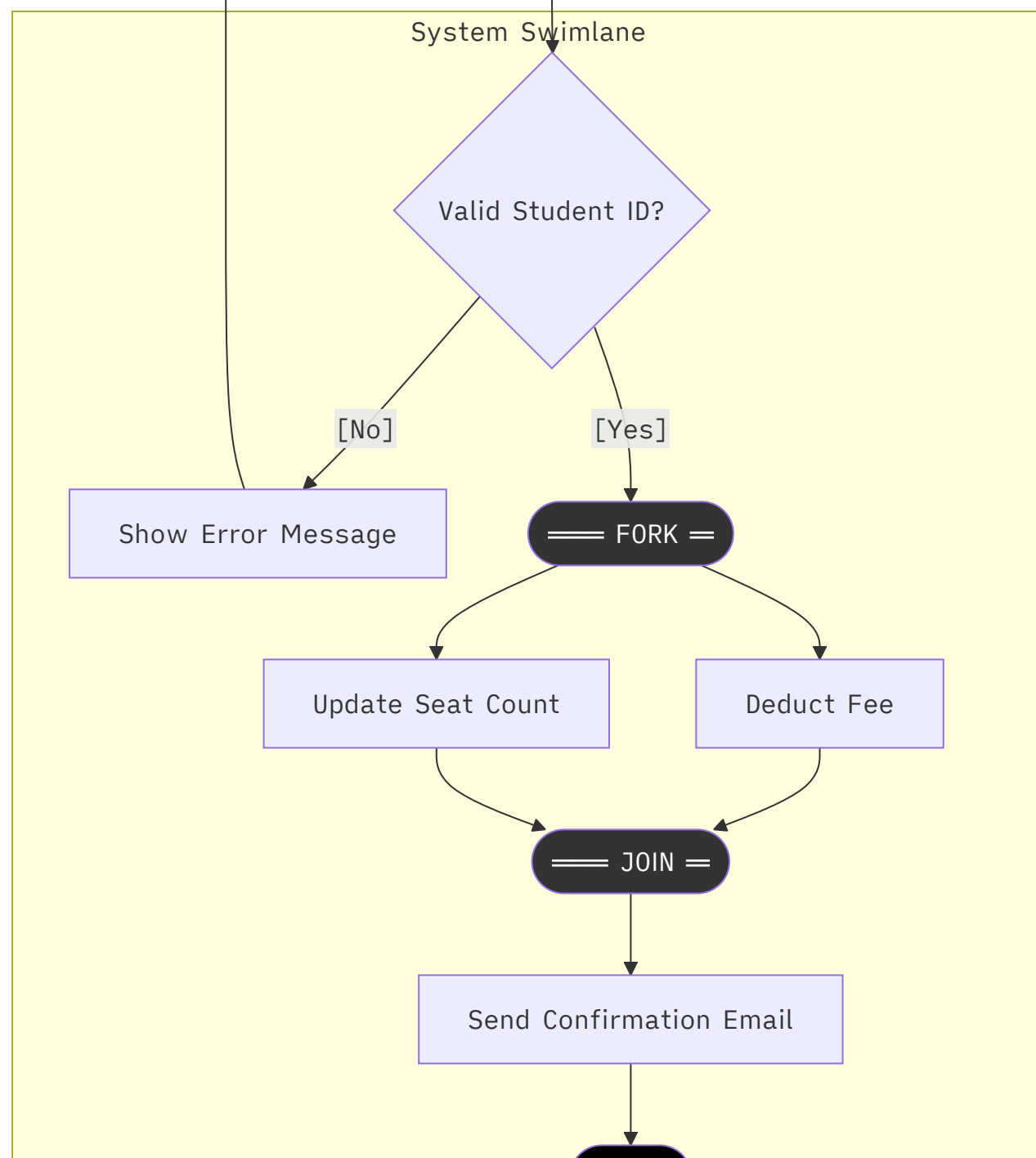
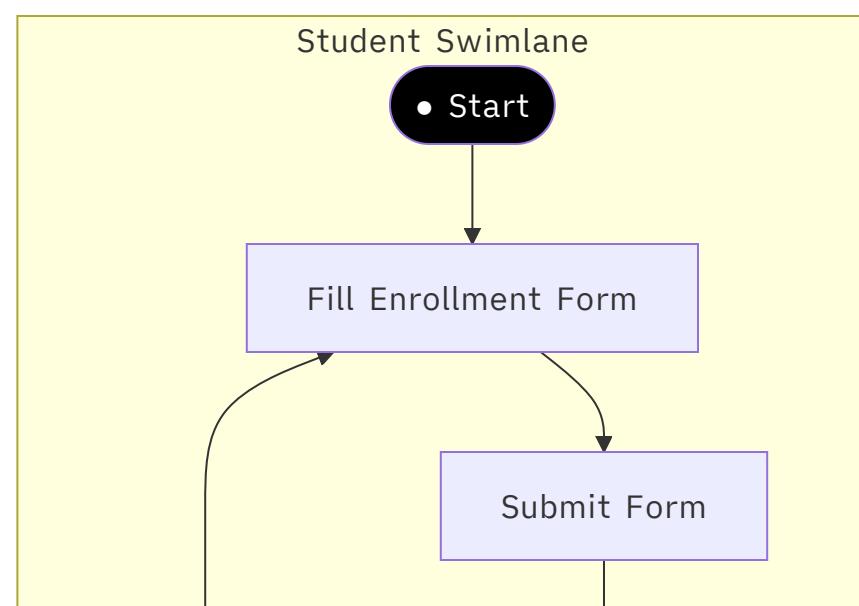
6. Common Mistakes Students Make:

- Forgetting the initial (black circle) and final (bullseye) nodes.
- Not writing guard conditions on Decision node arrows.
- Using Fork for decisions (Fork is ONLY for parallel; Decision is for if-else).
- Missing the Join after a Fork (every Fork must have a matching Join).

8. Diagram Drawing Rules:

1. Start with ● (Initial Node) — exactly one.
2. Use rounded rectangles for activities.
3. Use diamonds for decisions, ALWAYS label the outgoing arrows.
4. Use thick bars for Fork/Join.
5. End with ⊙ (Activity Final).
6. If using swimlanes, draw vertical columns and place activities in the correct lane.

 Visual: Activity Diagram — Course Enrollment with Swimlanes



💡 **Exam tip:** Fork and Join are represented as thick black bars in actual UML. Label them FORK and JOIN. Every Fork MUST have a matching Join.

12. Oral Trap Questions:

- Examiner:* "Can a Decision node have more than 2 outgoing arrows?"
- Answer:* "Yes, sir. A decision node can have multiple outgoing paths, one for each condition. But the conditions must be mutually exclusive and cover all cases."
- Examiner:* "What happens if one parallel path in a Fork finishes but the other doesn't?"
- Answer:* "The Join waits. It does not proceed until ALL parallel paths have completed — that is the purpose of synchronization."

Topic 2: Sequence Diagram (CRITICAL FOR PRACTICAL)

1. Simple Exam Definition:

A Sequence Diagram shows how objects **interact over time** — specifically, the **order of messages** exchanged between objects to complete a use case.

2. Core Concept Explanation — All Key Elements:

Element	Description
Object Box	Rectangle at the top: <code>objectName:ClassName</code>
Lifeline	Dotted vertical line — existence of object over time
Activation Bar	Narrow rectangle on lifeline — object is active/executing
Synchronous Message	Filled arrowhead → (caller waits for return)
Asynchronous Message	Open arrowhead → (caller does not wait)
Return Message	Dashed arrow with open head ← (reply from callee)
Delete Object	× at the bottom of lifeline — object is destroyed
Self-call	Arrow looping back to same object
Combined Fragment	Box around messages: <code>alt</code> , <code>opt</code> , <code>loop</code> , <code>ref</code> , <code>par</code>

Combined Fragments (Examiners love asking!):

- `alt` — if-else (like decision in activity diagram).
- `opt` — optional block (executes only if condition true).
- `loop` — repetition.
- `par` — parallel messages.
- `ref` — reference to another sequence diagram.

3. Real Practical Example (Student Management System — View Grades):

- `Student` sends `requestGrades(studentId)` to `GradesPortal`.

2. `GradesPortal` sends `fetchGrades(studentId)` to `GradeService`.
3. `GradeService` sends `queryDB(studentId)` to `Database`.
4. `Database` returns `ResultSet` to `GradeService`.
5. `GradeService` returns `gradesData[]` to `GradesPortal`.
6. `GradesPortal` returns `displayGrades(gradesData)` to `Student`.

4. Examiner Viva Questions:

- What is a Lifeline?
- What is the difference between synchronous and asynchronous messages?
- What is an Activation Bar?
- Can you show loops in a Sequence Diagram?

5. Strong Viva Answers:

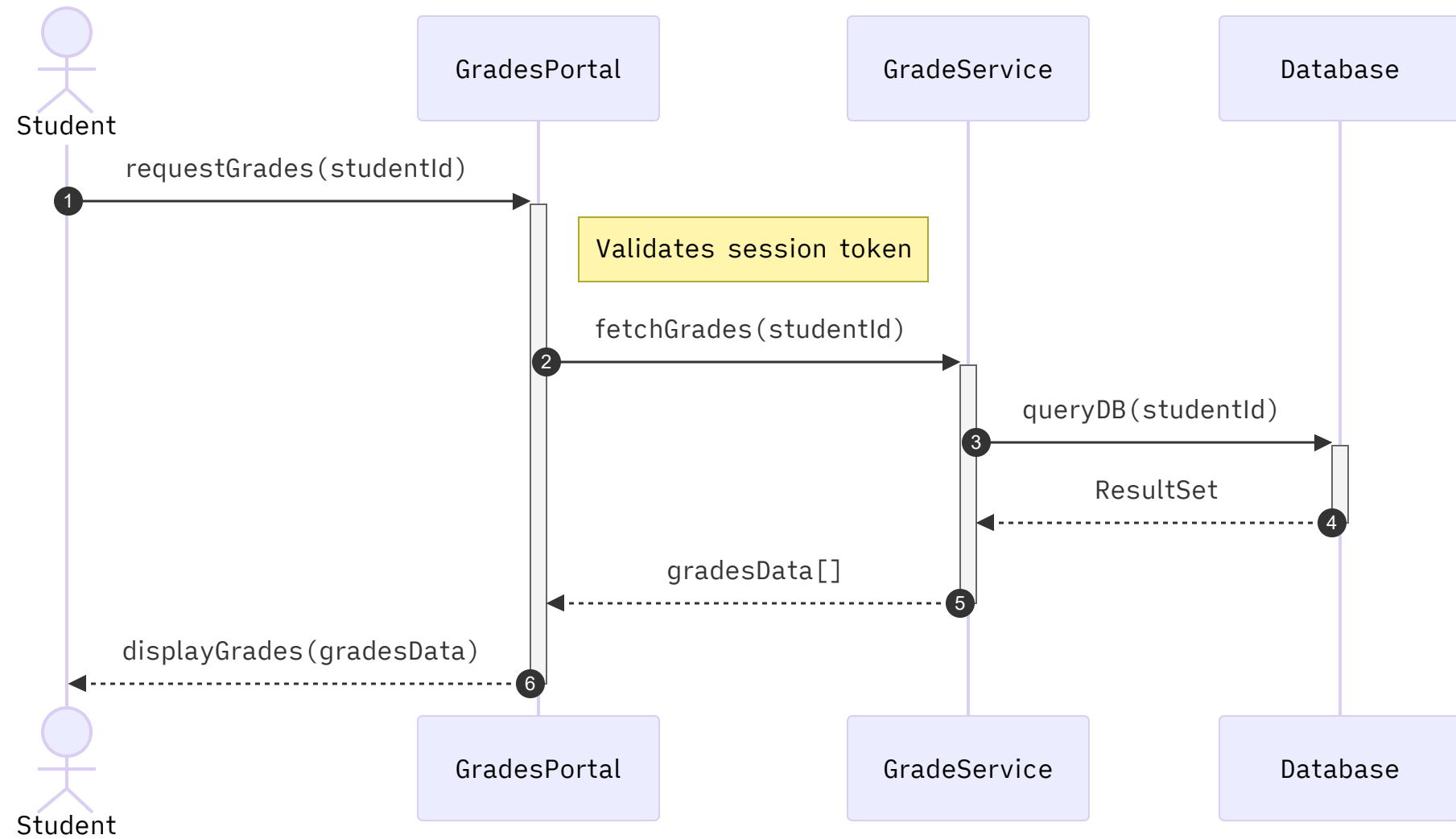
- *Answer:* A Lifeline is a dotted vertical line below an object, representing the object's existence over time.
- *Answer:* Synchronous messages use a filled arrowhead — the sender waits for a return. Asynchronous messages use an open arrowhead — the sender continues without waiting.
- *Answer:* An Activation Bar is a narrow rectangle on a lifeline showing the period when an object is actively processing.
- *Answer:* Yes, using the `loop` Combined Fragment box around the repeated messages.

6. Common Mistakes Students Make:

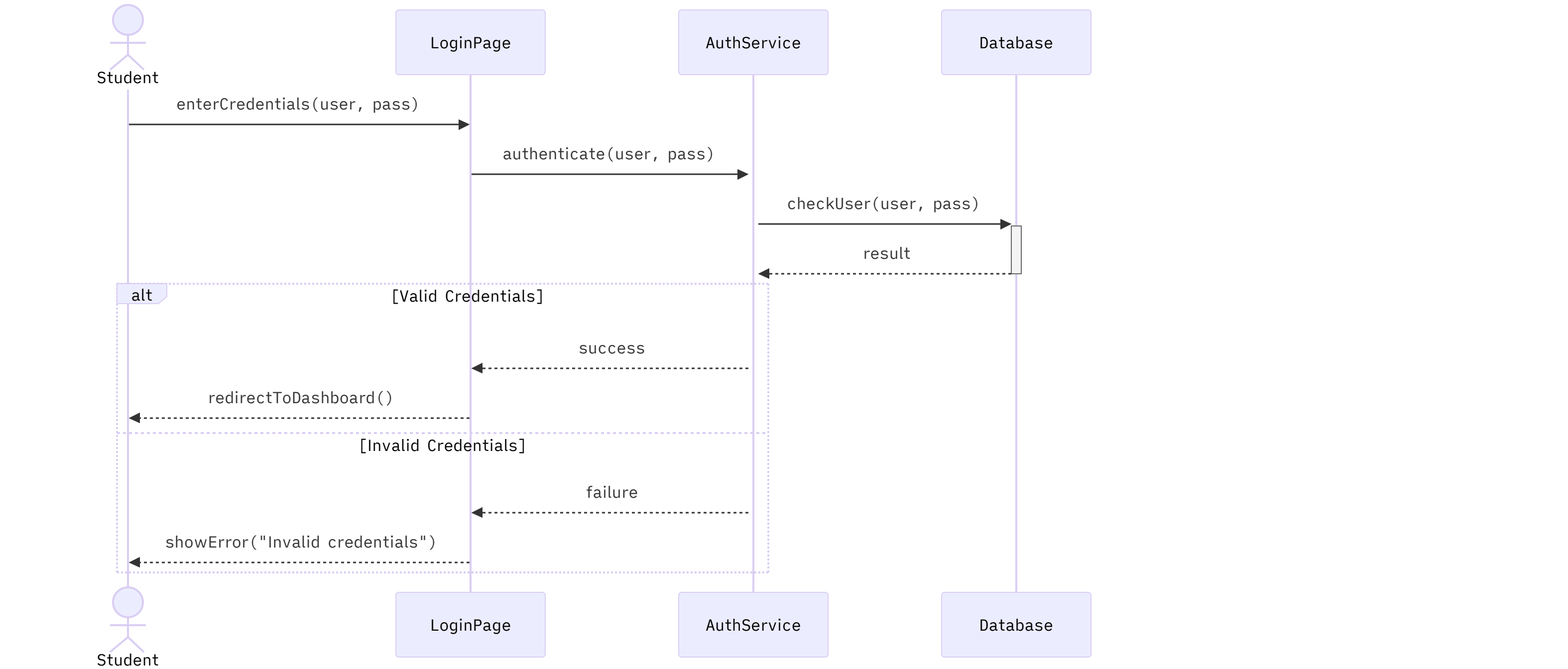
- Not labeling messages with method names.
- Drawing all messages at the same horizontal level (time should flow downward).
- Forgetting return messages (dashed arrows back).
- Not drawing activation bars.

8. Diagram Drawing Rules:

1. Objects at the top in boxes (`name:Class`).
2. Time flows **top to bottom**.
3. Solid arrows = calls/requests.
4. Dashed arrows = returns/replies.
5. Label every arrow with a method name.
6. Show activation bars as narrow rectangles.



 **Visual: Sequence Diagram with Alt Fragment — Login**



💡 **Exam tip:** The **alt** fragment is drawn as a box with a dashed line separating the if and else blocks. Label it with guard conditions like **[credentials valid]**.

12. Oral Trap Questions:

- *Examiner:* "Sequence diagram vs Communication diagram – what's the difference?"
- *Answer:* "Both show object interactions. Sequence diagrams focus on **time ordering**. Communication (Collaboration) diagrams focus on **structural relationships** and show numbered messages between objects without explicit time flow."

Topic 3: Communication (Collaboration) Diagram

1. Simple Exam Definition:

A Communication Diagram shows how objects collaborate by passing messages – similar to a Sequence Diagram, but emphasizes the **structural links** between objects rather than time ordering.

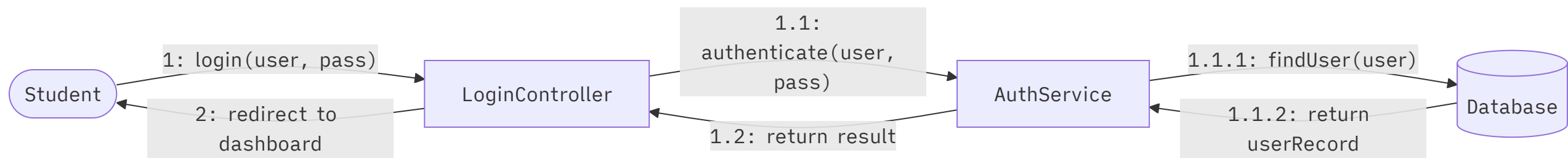
2. Core Concept Explanation:

- Objects are shown as rectangles (not arranged in a timeline).
- Links between objects are shown as solid lines.
- Messages are numbered to show sequence (e.g., 1: login() , 2: authenticate()).
- Numbering can be nested: 1.1 , 1.2 , 2.1 to show sub-messages.

3. Real Practical Example (Login Flow):

- Student → 1: login(user, pass) → LoginController
- LoginController → 1.1: authenticate(user, pass) → AuthService
- AuthService → 1.1.1: findUser(user) → Database

Visual: Communication Diagram — Login



4. Examiner Viva Questions:

- How does a Communication Diagram differ from a Sequence Diagram?
- How are messages ordered in a Communication Diagram?

5. Strong Viva Answers:

- Answer: A Sequence Diagram uses a vertical timeline showing the temporal order of messages. A Communication Diagram uses numbered messages on a structural link diagram, focusing on object relationships.

9. Difference Tables:

Feature	Sequence Diagram	Communication Diagram
Focus	Time ordering	Structural relationships
Layout	Timeline (top to bottom)	Network/graph layout
Sequence shown	By vertical position	By message numbers
Better for	Complex flows	Object collaboration overview

Topic 4: State Machine Diagram

1. Simple Exam Definition:

A State Machine Diagram shows the **lifecycle of a single object** — all the states it can be in, and the transitions between states triggered by events.

2. Core Concept Explanation:

Element	Description
State	A condition or situation of an object (rounded rectangle)
Initial Pseudostate	Black dot ● — entry point
Final State	Bullseye ⊙ — exit point
Transition	Arrow between states
Trigger/Event	What causes the transition (label on arrow)
Guard	Condition in [brackets] — transition fires only if true
Action	What happens during the transition /action
Composite State	A state that contains sub-states (nested states)

Transition Syntax:

Event [Guard] / Action

Example: payFee [amount > 0] / sendReceipt

Nested / Composite State:

A state can contain its own state machine. Example: Enrolled state contains sub-states ActiveStudent and OnLeave .

3. Real Practical Example (Student Account Lifecycle):

- New → Register → PendingVerification
- PendingVerification → verify → Active
- Active → suspend [violations > 3] → Suspended
- Suspended → appeal → Active
- Active → graduate → Alumni
- Alumni → ⊙ End

4. Examiner Viva Questions:

- What is the difference between State and Activity?
- What is a Composite State?
- What is a Guard in a State Diagram?

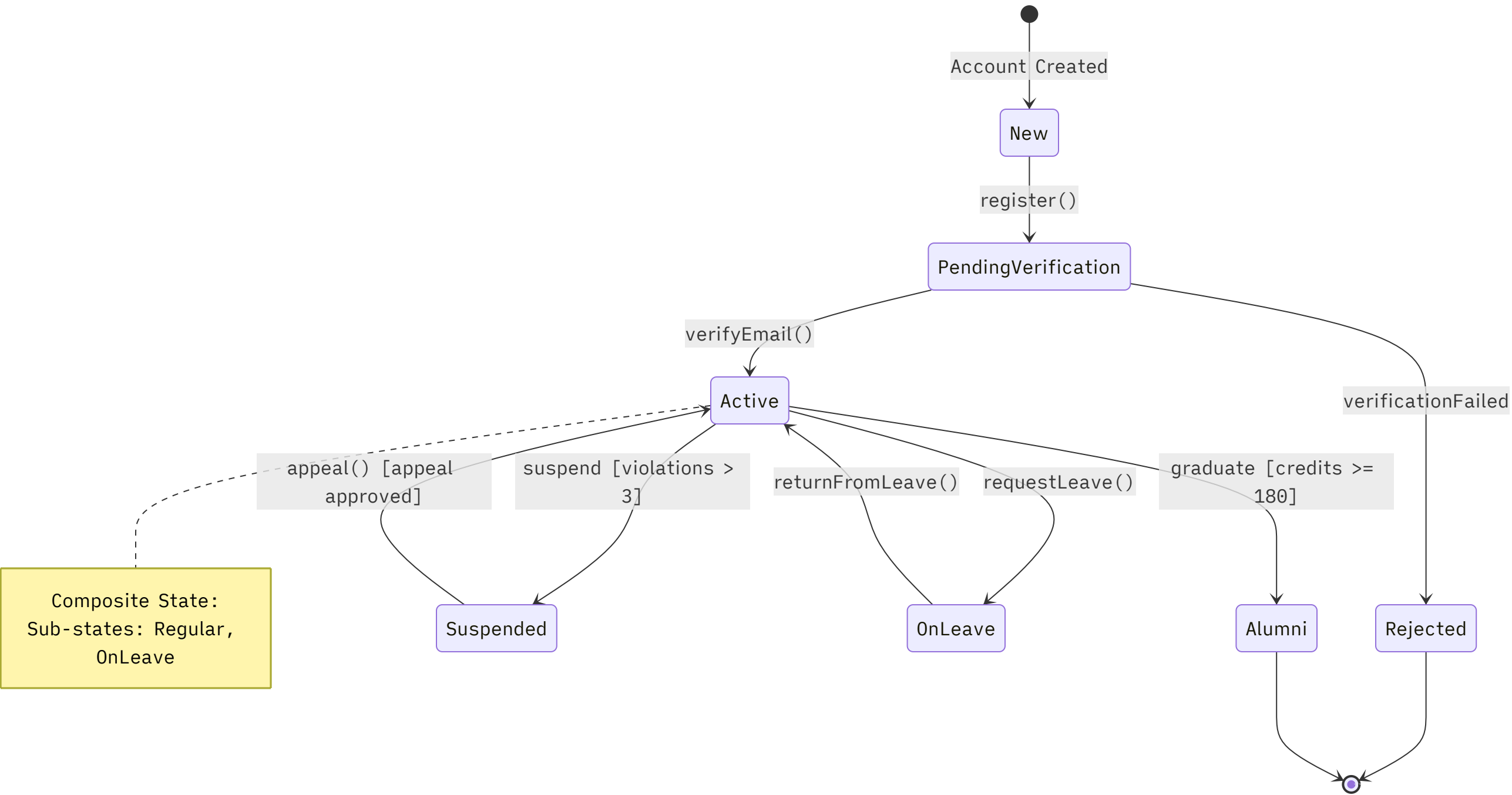
5. Strong Viva Answers:

- Answer: A State represents a **condition** of an object (a noun — e.g., "Active"). An Activity in an Activity Diagram represents an **action being performed** (a verb — e.g., "Process Payment").
- Answer: A Composite State is a state that contains nested sub-states, allowing complex lifecycle modeling within a single state.
- Answer: A Guard is a boolean condition in square brackets on a transition arrow. The transition fires only if the guard is true.

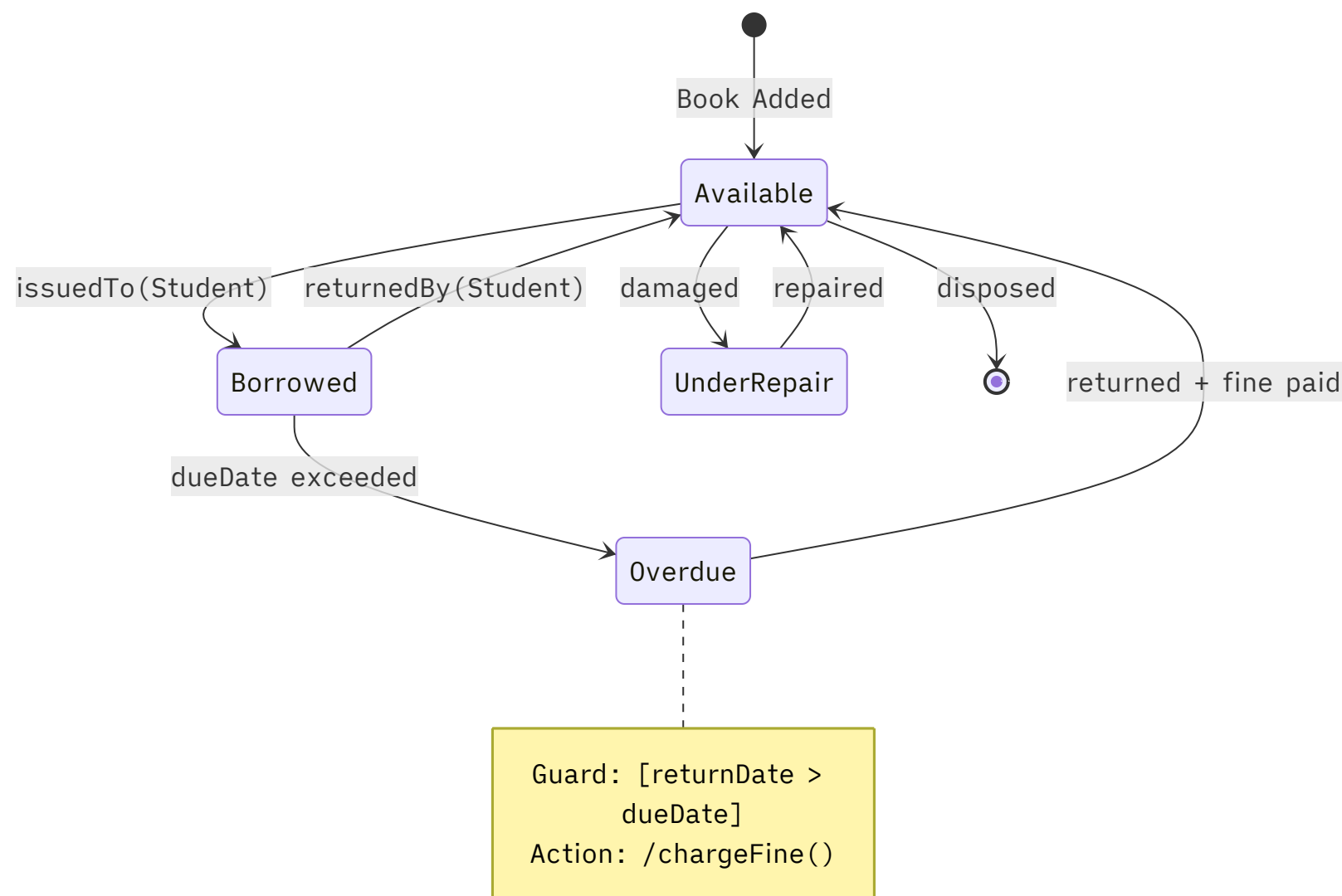
6. Common Mistakes Students Make:

- Drawing states for the entire system instead of one specific object.
- Forgetting the initial (●) and final (⊙) nodes.
- Not labeling transitions with events.

Visual: State Diagram — Student Account Lifecycle



Visual: State Diagram — Library Book Lifecycle (Alternate Example)



💡 **Exam tip:** Draw State diagrams for ONE object only. Popular exam questions: "Draw state diagram for Library Book lifecycle" or "Draw state diagram for Student Account."

10. Memory Tricks:

- State = **Noun** (Active, Enrolled, Suspended).
- Event = **Verb** (enroll, suspend, graduate).
- Guard = **Condition** `[credits ≥ 180]`.
- Action = **/Action** `/sendCertificate`.

Topic 5: Timing Diagram (Quick Revision)

1. Simple Exam Definition:

A Timing Diagram shows how the **state of an object changes over time** — specifically the exact time moments when transitions occur. Used in real-time and embedded systems.

2. Core Concept Explanation:

- X-axis = time.
- Y-axis = states.

- The line shows which state the object is in at each point in time.
- Useful for hardware timing, embedded systems, network protocols.

3. Real Practical Example:

For a `Student` object:

- Time 0–5 min: State = `LoggedOut`
- Time 5 min: `login()` event
- Time 5–30 min: State = `Active`
- Time 30 min: `timeout()` event
- Time 30+ min: State = `LoggedOut`

4. Examiner Viva Questions:

- What is a Timing Diagram used for?
- How is it different from a State Diagram?

5. Strong Viva Answers:

- *Answer:* A Timing Diagram shows state changes with precise time on the X-axis. A State Diagram shows possible states and transitions without time information.

 Visual: Timing Diagram Concept

```
Error parsing Mermaid diagram!

Invalid date:t0
```

 **Note:** A true UML Timing Diagram uses a waveform-style chart. For exams, a simplified text-based or Gantt-style description is acceptable.